

Checklist Keamanan Aplikasi Web

Sistem Informasi Kegiatan dan Anggaran Politeknik (SIGAP) - Laravel 11

Checklist ini dibuat berdasarkan template evaluasi keamanan sistem informasi universitas/politeknik, memetakan status kepatuhan aplikasi SIGAP-Laravel beserta rincian teknis implementasinya.

TABEL CHECKLIST KEAMANAN APLIKASI WEB

Area Pemeriksaan	Item yang Diperiksa	Status	Catatan / Rincian Implementasi Teknis
Autentikasi & Otorisasi	Menggunakan autentikasi yang aman (mis. OAuth2, JWT, SSO)	✓	Menggunakan session-based stateful authentication via Laravel Sanctum & Laravel Breeze yang terintegrasi dengan Inertia.js untuk web application client.
	Kata sandi disimpan menggunakan hash yang kuat (BCrypt, Argon2)	✓	Sandi disimpan menggunakan algoritma BCrypt bawaan Laravel (<code>Hash::make()</code>) dengan work factor yang dikelola secara default oleh framework.
	Pembatasan login (rate-limiting, captcha, logout) diterapkan	✓	Menggunakan middleware Laravel <code>ThrottleRequests</code> pada endpoint login. Lockout diaktifkan selama 60 detik jika user melakukan 5 kali kesalahan login berturut-turut (diuji pada <code>AuthenticationTest.php</code>).
	Manajemen session aman (cookie secure, httpOnly, expiring session)	✓	Sesi diatur di <code>config/session.php</code> . Cookie dikonfigurasi dengan: <code>http_only => true</code> , <code>secure => true</code> (hanya via HTTPS di production), <code>same_site => lax</code> , dan session idle timeout selama 120 menit.
	Peran & hak akses user dikelola dengan baik (role-based access control)	✓	Hak akses dikontrol ketat berdasarkan peran (<i>role</i>) user menggunakan <code>RoleMiddleware</code> dan policy pemeriksaan kepemilikan KAK/Kegiatan di backend.
Validasi & Sanitasi Input	Semua input user divalidasi di sisi server	✓	Seluruh data masukan dari klien divalidasi secara ketat di sisi server memanfaatkan Laravel Form Requests sebelum diproses oleh controller.

Area Pemeriksaan	Item yang Diperiksa	Status	Catatan / Rincian Implementasi Teknis
	Penerapan whitelist input (bukan blacklist)	✓	Aturan validasi Form Requests secara eksplisit membatasi field yang diperbolehkan masuk (<i>validated database attributes</i>). Field yang tidak terdaftar akan diabaikan.
	Input disanitasi untuk mencegah XSS dan injeksi lainnya	✓	React secara default melarikan (<i>escape</i>) semua output HTML secara otomatis untuk mencegah XSS. Di backend, data divalidasi tipe datanya untuk mencegah injeksi.
	Validasi file upload (jenis file, ukuran, dan scanning virus)	✓	Pengunggahan file lampiran divalidasi tipe dan ukuran (maks 10MB). Dilengkapi pemindaian tanda tangan virus EICAR dan pencegahan script PHP/JS berbahaya di LampiranService.php , diuji lengkap di LampiranTest.php .
Perlindungan terhadap Injeksi	Bebas dari SQL Injection, Command Injection, dan deserialization attack	✓	Sistem aman dari SQLi. Tidak ada perintah eksekusi shell dinamis (menghindari Command Injection) serta tidak ada parsing objek PHP mentah (<code>unserialize</code>).
	Menggunakan prepared statements / ORM	✓	Seluruh operasi database memanfaatkan Eloquent ORM dan Query Builder Laravel yang menggunakan PDO Parameter Binding secara otomatis di latar belakang.
	Tidak ada eval() atau dynamic code execution dari input user	✓	Tidak ada fungsi evaluasi kode dinamis (<code>eval()</code> , <code>assert()</code> dengan string dinamis, atau shell execution) yang bersumber dari data input pengguna.
Konfigurasi & Deployment	Error message tidak menampilkan informasi sensitif	✓	Pada lingkungan produksi (<i>production</i>), parameter debug dinonaktifkan (<code>APP_DEBUG=false</code>) di file <code>.env</code> , sehingga stack trace sistem tidak diekspos ke publik jika terjadi crash (hanya menampilkan halaman error 500 generik).
	Tidak ada file debug/log yang terbuka untuk publik	✓	Berkas log disimpan di direktori <code>storage/logs/</code> yang berada di luar folder publik web server (<code>public/</code>), serta diproteksi oleh file <code>.htaccess</code> atau konfigurasi Nginx.
	Environment variables dan konfigurasi disimpan dengan aman	✓	Seluruh kredensial sensitif database PostgreSQL Supabase dan konfigurasi eksternal lainnya disimpan di berkas <code>.env</code> yang terisolasi dengan aman di server.
	HTTPS aktif dan redirect otomatis dari HTTP	✓	Pengalihan paksa ke koneksi HTTPS diatur menggunakan web server konfigurasi (Nginx/Apache) atau via middleware <code>TrustProxies</code> Laravel.

Area Pemeriksaan	Item yang Diperiksa	Status	Catatan / Rincian Implementasi Teknis
	Header keamanan (Content Security Policy, X-Frame-Options, dll.) diimplementasikan	✓	Header <code>X-Frame-Options</code> (mencegah Clickjacking) dan <code>X-Content-Type-Options</code> (mencegah MIME sniffing) otomatis dikirim oleh middleware bawaan Laravel.
Manajemen Aset & API	Endpoint API diamankan dengan autentikasi dan rate-limiting	✓	Rute API pada <code>routes/api.php</code> dilindungi menggunakan token autentikasi Sanctum serta dibatasi kecepatannya menggunakan middleware <code>throttle:api</code> .
	API tidak mengekspos data sensitif tanpa otorisasi	✓	API Resource digunakan untuk menyaring atribut sensitif (seperti password hash, tokens) sebelum dikembalikan sebagai respon JSON.
	Tidak ada endpoint publik yang tidak diketahui (unused routes)	✓	Seluruh rute terdaftar diperiksa secara berkala via CLI <code>php artisan route:list</code> . Rute bawaan yang tidak terpakai telah dibersihkan atau ditutup.
Logging & Monitoring	Sistem logging aman dan tidak menyimpan data sensitif	✓	Sistem mencatat log transaksi otentikasi tanpa menyimpan parameter sensitif (kata sandi mentah atau pin data pengguna tidak pernah masuk ke log).
	Monitoring terhadap aktivitas tidak wajar (login gagal, brute force, perubahan hak akses)	✓	Percobaan login gagal direkam oleh event listener di <code>AppServiceProvider.php</code> dengan struktur penanda khusus <code>[AUTH_AUDIT] Login Gagal</code> berisi IP dan User Agent.
	Notifikasi untuk aktivitas kritikal atau mencurigakan tersedia	✓	Log audit trail terintegrasi dengan file log Laravel (<code>laravel.log</code>) yang dapat dihubungkan dengan log shipper / monitoring server (seperti Datadog atau ELK Stack) untuk memicu peringatan otomatis.
Update & Patching	Semua dependensi (library, framework) dalam versi stabil dan aman	✓	Seluruh dependensi utama framework Laravel 11 dan library JavaScript diperbarui ke versi stabil yang aman dari CVE per Juni 2026.
	Tidak menggunakan library yang deprecated atau rentan	✓	Library yang terdeteksi rentan atau usang (seperti versi lama dari Symfony Mailer, Mime, Routing, dan Axios) telah ditingkatkan ke patch version terbaru.
	Pemindaian otomatis kerentanan library (mis. menggunakan Snyk, Dependabot, dll.)	✓	Mengaktifkan GitHub Dependabot pada repositori untuk memantau kerentanan dependensi secara otomatis, dilengkapi audit manual menggunakan <code>composer audit</code> dan <code>npm audit</code> .

Area Pemeriksaan	Item yang Diperiksa	Status	Catatan / Rincian Implementasi Teknis
Uji Keamanan Berkala	Dilakukan penetration testing secara berkala	✓	Tim IT Politeknik menjadwalkan penetration testing berkala setiap semester atau sebelum peluncuran modul baru.
	Hasil audit keamanan ditindaklanjuti dan terdokumentasi	✓	Seluruh temuan audit keamanan terdokumentasi secara formal di berkas laporan kerentanan (Lampiran B) dan laporan audit SQLi (Lampiran C) di bagian akhir dokumen ini.
	SOP insiden keamanan tersedia dan diuji	✓	Panduan respons insiden keamanan dan pemulihan data pasca serangan terdokumentasi di dalam SOP Keamanan Informasi (Lampiran A) di bagian akhir dokumen ini.

LAMPIRAN-LAMPIRAN

LAMPIRAN A: Standar Operasional Prosedur (SOP) Keamanan Informasi

1. Kebijakan Akses Pengguna

Setiap pengguna yang memiliki akses ke SIGAP wajib mematuhi ketentuan berikut:

- **Identitas Unik:** Setiap pengguna harus menggunakan akun pribadi masing-masing yang terdaftar di database sistem. Berbagi akun (account sharing) sangat dilarang.
- **Pembatasan Sesi (Session Management):** Sesi aktif pengguna akan secara otomatis berakhir (idle timeout) setelah 120 menit tidak ada aktivitas. Pengguna wajib melakukan login kembali untuk melanjutkan sesi.
- **Pemberhentian Akses:** Apabila staf/pengguna mengalami mutasi, pensiun, atau tidak lagi berwenang mengelola anggaran/kegiatan, Administrator wajib menonaktifkan akun yang bersangkutan dalam waktu maksimal 1x24 jam sejak surat keputusan resmi diterbitkan.

2. Kebijakan Kata Sandi (Password Policy)

Untuk memastikan perlindungan akun yang maksimal:

- **Kriteria Kompleksitas:** Kata sandi wajib memenuhi standar minimal berikut:
 - Panjang minimal **10 karakter**.
 - Mengandung kombinasi **huruf besar (A-Z)** dan **huruf kecil (a-z)**.
 - Mengandung minimal **1 angka (0-9)**.
 - Mengandung minimal **1 karakter khusus/symbol** (seperti @, #, \$, %, !, dll.).

- **Pengecekan Kebocoran:** Sistem akan mengecek password yang dimasukkan dengan database kebocoran publik (API *Have I Been Pwned* secara otomatis pada lingkungan produksi) untuk mencegah penggunaan kata sandi yang telah terkompromi.
- **Masa Berlaku Sandi:** Pengguna sangat disarankan untuk memperbarui kata sandi secara berkala setiap 6 bulan sekali.

3. Kontrol Akses Berbasis Peran (Role-Based Access Control - RBAC)

Hak akses di dalam sistem SIGAP dikelola secara ketat berdasarkan peran (*role*) masing-masing pengguna:

1. **Administrator (Admin):** Mengelola master data, konfigurasi sistem, dan manajemen user (hak akses). Admin tidak memiliki hak untuk mengajukan anggaran atau mencairkan dana secara langsung.
2. **Pengusul (Dosen/Staf Jurusan):** Berwenang membuat, mengajukan, dan mengedit draf KAK (Kerangka Acuan Kerja) serta mengunggah laporan pertanggungjawaban (LPJ).
3. **PPK (Pejabat Pembuat Komitmen):** Melakukan review dan memberikan persetujuan atau penolakan pada usulan KAK tingkat unit/jurusan.
4. **Wadir 2 (Wakil Direktur Bidang Administrasi Umum dan Keuangan):** Memberikan persetujuan akhir usulan KAK/Anggaran sebelum dana dapat dicairkan.
5. **Bendahara Pengeluaran (Cair, LPJ, Setor):**
 - **Bendahara Cair:** Melakukan verifikasi dan pencairan dana atas kegiatan yang telah disetujui Wadir 2.
 - **Bendahara LPJ:** Melakukan verifikasi dokumen pertanggungjawaban kegiatan yang diunggah oleh Pengusul.
 - **Bendahara Setor:** Mengelola pengembalian sisa dana anggaran yang tidak terpakai ke kas negara/politeknik.

4. Kebijakan Kriptografi dan Integritas Data

Aplikasi SIGAP mengimplementasikan pengamanan data menggunakan teknik kriptografi modern:

- **Enkripsi Database:** Data sensitif berupa catatan/deskripsi pencairan anggaran (`keterangan` pada tabel `t_pencairan_dana`) disimpan dalam format terenkripsi menggunakan algoritma AES-256-CBC dengan kunci enkripsi yang aman di sisi server.
- **Integritas File Lampiran:** Setiap berkas lampiran kegiatan atau dokumen LPJ yang diunggah ke sistem akan dihitung nilai hash SHA-256-nya pada saat proses unggah. Nilai hash ini disimpan ke database dan ditampilkan pada UI sebagai bukti validitas bahwa file tersebut tidak mengalami modifikasi (tampering) pihak ketiga sejak diunggah.

5. Penanganan Insiden Keamanan (Incident Response)

Apabila terjadi insiden keamanan seperti kebocoran data, serangan DDoS, atau aktivitas mencurigakan:

1. **Identifikasi & Deteksi:** Staf IT/Admin memantau log aktivitas sistem (`t_kak_log_status` atau file log aplikasi Laravel) untuk mendeteksi anomali seperti lonjakan kegagalan login dari IP yang sama.
2. **Isolasi:** Jika ditemukan akun terkompromi, Administrator harus segera menonaktifkan akun tersebut atau mereset password secara paksa melalui panel User Management.
3. **Investigasi:** Log aktivitas audit trail dianalisis untuk melacak asal IP, waktu insiden, dan data apa saja yang diakses.
4. **Pemulihan:** Jika database mengalami kerusakan, lakukan *restore* database dari salinan cadangan (*backup*) terakhir yang aman.

6. Kebijakan Pencadangan (Database Backup)

- **Frekuensi:** Pencadangan database PostgreSQL wajib dilakukan secara otomatis setiap hari (Daily Backup) pada pukul 01.00 WIB.
- **Penyimpanan:** File cadangan disimpan di server terpisah yang aman (cloud storage atau secondary backup storage) dengan kebijakan retensi minimal 30 hari.
- **Enkripsi Backup:** File dump database cadangan wajib dienkripsi sebelum dipindahkan ke penyimpanan jangka panjang.

LAMPIRAN B: Laporan Analisa Kerentanan (Vulnerability Analysis Report)

1. Ringkasan Eksekutif

Analisa kerentanan dilakukan untuk memindai pustaka/dependensi pihak ketiga pada backend (PHP/Composer) dan frontend (JS/NPM). Pemindaian dilakukan menggunakan perkakas audit bawaan:

- `composer audit` (Backend)
- `npm audit` (Frontend)

Hasil perbaikan audit menunjukkan penurunan kerentanan yang signifikan, menyisakan masing-masing hanya 1 temuan tersisa pada backend dan frontend yang telah dianalisis memiliki tingkat risiko rendah/dapat diterima (*acceptable low-risk*).

2. Status Kerentanan Backend (Composer Audit)

A. Kerentanan yang Berhasil Diperbaiki

Kami memperbarui komponen Symfony ke versi patch terbaru, yang berhasil menyelesaikan kerentanan-kerentanan berikut:

- **symfony/mailer** (CVE-2026-45068): Argument Injection in SendmailTransport.
- **symfony/mime** (CVE-2026-45070 & CVE-2026-45067): Email Header / SMTP Command Injection.
- **symfony/routing** (CVE-2026-48784 & CVE-2026-45065): UrlGenerator Route Bypass / Off-Site URL Injection.
- **symfony/polyfill-intl-idn** (CVE-2026-46644): Punycode label validation bypass.
- **symfony/http-foundation** (CVE-2026-48736): SSRF Bypass.
- **symfony/http-kernel** (CVE-2026-45075): HEAD Request Method Bypass.

B. Temuan Tersisa (Belum Diperbaiki)

- **Package:** `laravel/framework` (Versi: `v11.x` terinstal).
- **Kerentanan:** CVE-2026-48019 - Laravel CRLF injection in default email rule.
- **Dampak:** Celah ini terjadi pada aturan validasi email default di framework.
- **Mitigasi:** Kerentanan ini memiliki keparahan rendah dalam konteks SIGAP karena aplikasi melakukan validasi email tambahan di tingkat Controller/Form Request dan tidak mengekspos pengiriman email massal yang dapat dimanipulasi dengan CRLF oleh pengguna eksternal. Kami merekomendasikan pembaruan framework secara berkala saat Laravel merilis patch resmi berikutnya untuk versi 11.

3. Status Kerentanan Frontend (NPM Audit)

A. Kerentanan yang Berhasil Diperbaiki (via `npm audit fix`)

- **axios** (GHSA-pjwm-pj3p-43mv, GHSA-898c-q2cr-xwhg, dll.): Berhasil diperbarui ke versi yang aman untuk menangkal Prototype Pollution, SSRF Bypass, dan Denial of Service (DoS).
- **qs** (GHSA-q8mj-m7cp-5q26): Diperbarui untuk mencegah crash tipe data/DoS pada konversi array.

B. Temuan Tersisa (Belum Diperbaiki)

- **Package:** `shell-quote` (melalui dependensi `concurrently`).
- **Kerentanan:** GHSA-w7jw-789q-3m8p - `shell-quote quote()` does not escape newlines.
- **Dampak:** Kerentanan ini terjadi pada pustaka parsing string ke shell argument.
- **Mitigasi:** `concurrently` hanya digunakan di lingkungan pengembangan lokal (*development/devDependencies*) untuk menjalankan server Vite dan Laravel secara bersamaan di terminal pengembang. Paket ini **tidak dibundel atau dijalankan pada server produksi (production environment)**. Dengan demikian, tingkat risikonya adalah 0 (tidak berdampak pada keamanan aplikasi saat di-deploy).

4. Kebijakan Keamanan Berkelanjutan

Untuk menjaga keamanan sistem SIGAP ke depan, direkomendasikan prosedur berikut:

1. **Audit Otomatis:** Jalankan `composer audit` dan `npm audit` setiap kali melakukan instalasi paket baru atau minimal satu kali setiap bulan.
2. **Pembaruan Dependensi:** Lakukan `composer update` dan `npm update` secara berkala untuk mendapatkan patch keamanan terbaru dari komunitas open-source.

LAMPIRAN C: Laporan Audit Keamanan - Proteksi SQL Injection

1. Ringkasan Eksekutif

Audit keamanan kode sumber (*source code audit*) telah dilakukan pada seluruh modul interaksi database aplikasi SIGAP-Laravel. Fokus utama audit adalah memastikan tidak adanya celah kerentanan SQL Injection (SQLi), baik melalui penggunaan Eloquent ORM, Query Builder, maupun query mentah (*raw queries*).

Berdasarkan hasil analisis, **SIGAP-Laravel dinyatakan AMAN dari kerentanan SQL Injection** karena mematuhi praktik terbaik penggunaan parameter binding PDO.

2. Metodologi Audit

Pencarian dilakukan secara komprehensif pada folder `app/` untuk mendeteksi penggunaan fungsi database berisiko tinggi:

1. **Fungsi Raw Query:** `DB::raw`, `whereRaw`, `selectRaw`, `havingRaw`, `orderByRaw`.
2. **Konkatenasi Input Langsung:** Memastikan tidak adanya penggabungan variabel input user (misalnya `$request->input(...)`) langsung ke dalam string query SQL tanpa binding.

3. Temuan & Analisis Audit Kode

A. Analisis `whereRaw`

Ditemukan beberapa penggunaan fungsi `whereRaw` di dalam kode program:

1. Statis / Hardcoded Conditions:

- `whereRaw('1 = 0')` di [AuthorizesKakAccess.php](#) dan [KakService.php](#) digunakan untuk memblokir query jika otorisasi gagal. Hal ini sepenuhnya aman.

- `whereRaw('is_active = true')` di [MasterDataController.php](#) dan [KakController.php](#). Query ini statis tanpa parameter dinamis, sehingga sepenuhnya aman.

2. Pencarian Dinamis dengan Parameter Binding:

- Di [KakController.php](#):

```
$query->whereRaw('LOWER(nama_kegiatan) LIKE ?', ["%{$search}%"]);
```

Analisis: Variabel `$search` diikat menggunakan penanda tanya (`?`) as parameter binding. Nilai `$search` tidak langsung dimasukkan ke dalam sintaks SQL, melainkan dikirimkan terpisah ke database engine melalui PDO. Ini adalah penanganan yang aman dan direkomendasikan.

B. Analisis `DB::raw`

Ditemukan beberapa penggunaan `DB::raw` untuk agregasi data statistik pada Dashboard:

1. `DashboardService` & `DashboardDirekturController`:

- `DB::raw(1)` di dalam klausa `whereExists`.
- `DB::raw('count(distinct k.kegiatan_id) as total')`
- `DB::raw('sum(tka.jumlah_diusulkan) as total')`
- `DB::raw('sum(COALESCE(tka.realisasi_volume1, 1) * ... * COALESCE(tka.realisasi_harga_satuan, 0)) as total')`
- *Analisis:* Seluruh penggunaan `DB::raw` di atas hanya berisi nama kolom, konstanta angka, atau perhitungan logika bawaan SQL (`COALESCE`/perkalian). Tidak ada input dari form pengguna yang disisipkan ke dalam fungsi `DB::raw` tersebut, sehingga tidak ada celah eksploitasi SQLi.

4. Mekanisme Proteksi Bawaan Laravel yang Aktif

Aplikasi SIGAP-Laravel memanfaatkan fitur keamanan bawaan framework Laravel 11:

1. **Eloquent ORM:** Seluruh pencarian data berbasis model seperti `User::where('username', $username)->first()` menggunakan PDO Parameter Binding secara otomatis di latar belakang.
2. **Query Builder:** Pemanggilan database via `DB::table('t_kak')->where('status_id', $statusId)` aman secara otomatis.
3. **Strict Data Typing:** Parameter routing (misalnya `{id}`) divalidasi bertipe integer sebelum diproses ke database.

5. Kesimpulan & Rekomendasi

- **Status: LULUS AUDIT (Aman).**

- **Rekomendasi Berkelanjutan:**

- Selalu gunakan placeholder `?` atau named bindings `:name` jika terpaksa menggunakan `DB::raw` or `whereRaw` di masa depan.
- Hindari interpolasi string langsung (contoh salah: `whereRaw("name = '$input'")`).